AI Research Paper Assistant Using Retrieval-Augmented Generation (RAG)

Project Report

1. Abstract

This application was created using Python to allow you to easily organise, search, understand and compare your research papers. This tool uses Retrieval Augmented Generation (RAG) and Google Gemini models to provide answers that are based on information extracted from your research papers. The application allows ingestion of documents, hybrid search, question answer using citations, creating literature reviews and checking references.

2. Objectives

The aim of this application is to save time by reducing manual reading of research papers, allow searching of multiple documents, create structured summaries and literature reviews. And provide answers that can be traced back to sources. This is meant to be used to assist with your research.

3. Methodology and Workflow

To start your research papers are imported in one of the supported formats (PDF, DOCX, EPUB, TXT, Markdown and HTML). The text and metadata are extracted using the parser. If there are scanned pages then Vision OCR will be used. The text is then chunked into overlapping paragraphs. The Gemini Embeddings will be used to create semantic representations. These are then stored locally in the SQLite database where full text search is also available. When a query is made, it will use both BM25 and vector similarity search. The results are combined using Reciprocal Rank Fusion to get the most relevant passages. These will then be passed to the Gemini model so a contextually aware answer can be provided with citations [Title, page X].

4. Technology

The application has been built using Python. This uses Streamlit for the front end, Google Gemini for text generation, embeddings and OCR. The paper's metadata, chunks, notes, conversations and keywords are stored in the SQLite database using FTS5. The content is extracted using PyMuPDF and document parsing libraries. The researchmind package has been used to separate functionality. The parser.py file is used to do text extraction, chunker.py creates overlapping chunks, ingest.py handles importing documents, db.py handles saving data, search.py does hybrid search, rag.py provides contextually aware answers and creates literature reviews, verify.py checks claims and references and export.py creates exportable outputs.

5. Outcome

The application allows users to manage their papers, query one or multiple papers, view citations, run critique or pro and con prompts, compare papers, search for similar literature, create a multi section literature review, take notes and export literature reviews. External scholarly sources (OpenAlex, Crossref, Semantic Scholar) are used to check DOIs and references. The application stores files and the database locally in the data directory. The text excerpts and prompts are sent to the Gemini API that has been set up. This project shows an end to end RAG application for academic research. Key features include tracing back to sources and hybrid search. The quality of the output will depend on how well the documents have been extracted, how relevant the search is, how the models behave and if the sources are valid. Therefore any generated content should be verified before use in academic contexts.